

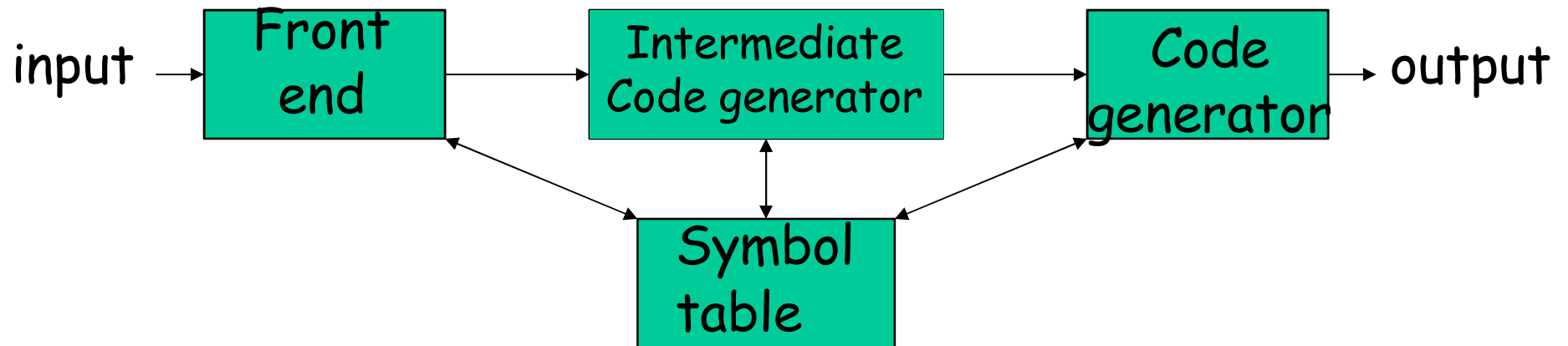
CO4: Transfer intermediate code into basic block and flow graphs to generate efficient target code. (CDL2)

Code Generation

Part - 01

D. VIJAYAKUMAR,
AP(SG) / CSE

Code generation and Instruction Selection



Requirements

- output code must be correct
- output code must be of high quality
- code generator should run efficiently

Design of code generator: Issues



- Input: Intermediate representation with symbol table
 - assume that input has been validated by the front end
- Target programs :
 - absolute machine language
fast for small programs
 - relocatable machine code
requires linker and loader
 - assembly code
requires assembler, linker, and loader

More Issues...



- Instruction selection
 - Uniformity
 - Completeness
 - Instruction speed, power consumption
- Register allocation
 - Instructions with register operands are faster
 - store long life time and counters in registers
 - temporary locations
 - Even odd register pairs
- Evaluation order

Instruction Selection

- straight forward code if efficiency is not an issue

a=b+c

Mov b, R₀

d=a+e

Add c, R₀

Mov R₀, a

Mov a, R₀

can be eliminated

Add e, R₀

Mov R₀, d

a=a+1

Mov a, R₀

Inc a

Add #1, R₀

Mov R₀, a

Example Target Machine

- Byte addressable with 4 bytes per word
- n registers $R_0, R_1, \dots, R_{n-1}$
- Two address instructions of the form
opcode source, destination
- Usual opcodes like move, add, sub etc.
- Addressing modes

MODE	FORM	ADDRESS
Absolute	M	M
register	R	R
index	$c(R)$	$c + \text{content}(R)$
indirect register	$*R$	$\text{content}(R)$
indirect index	$*c(R)$	$\text{content}(c + \text{content}(R))$
literal	$\#c$	c

Flow Graph



- Graph representation of three address code
- Useful for understanding code generation (and for optimization)
- Nodes represent computation
- Edges represent flow of control

Basic blocks

- (maximum) sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end

Algorithm to identify basic blocks

- determine leader
 - first statement is a leader
 - any target of a goto statement is a leader
 - any statement that follows a goto statement is a leader
- for each leader its basic block consists of the leader and all statements up to next leader

Example

```
1) i = 1
2) j = 1
3) t1 = 10 * i
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1
11) if i <= 10 goto (2)
12) i = 1
13) t5 = i - 1
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

```
for i from 1 to 10 do
  for j from 1 to 10 do
    a[i,j]=0.0

for i from 1 to 10 do
  a[i,i]=0.0
```

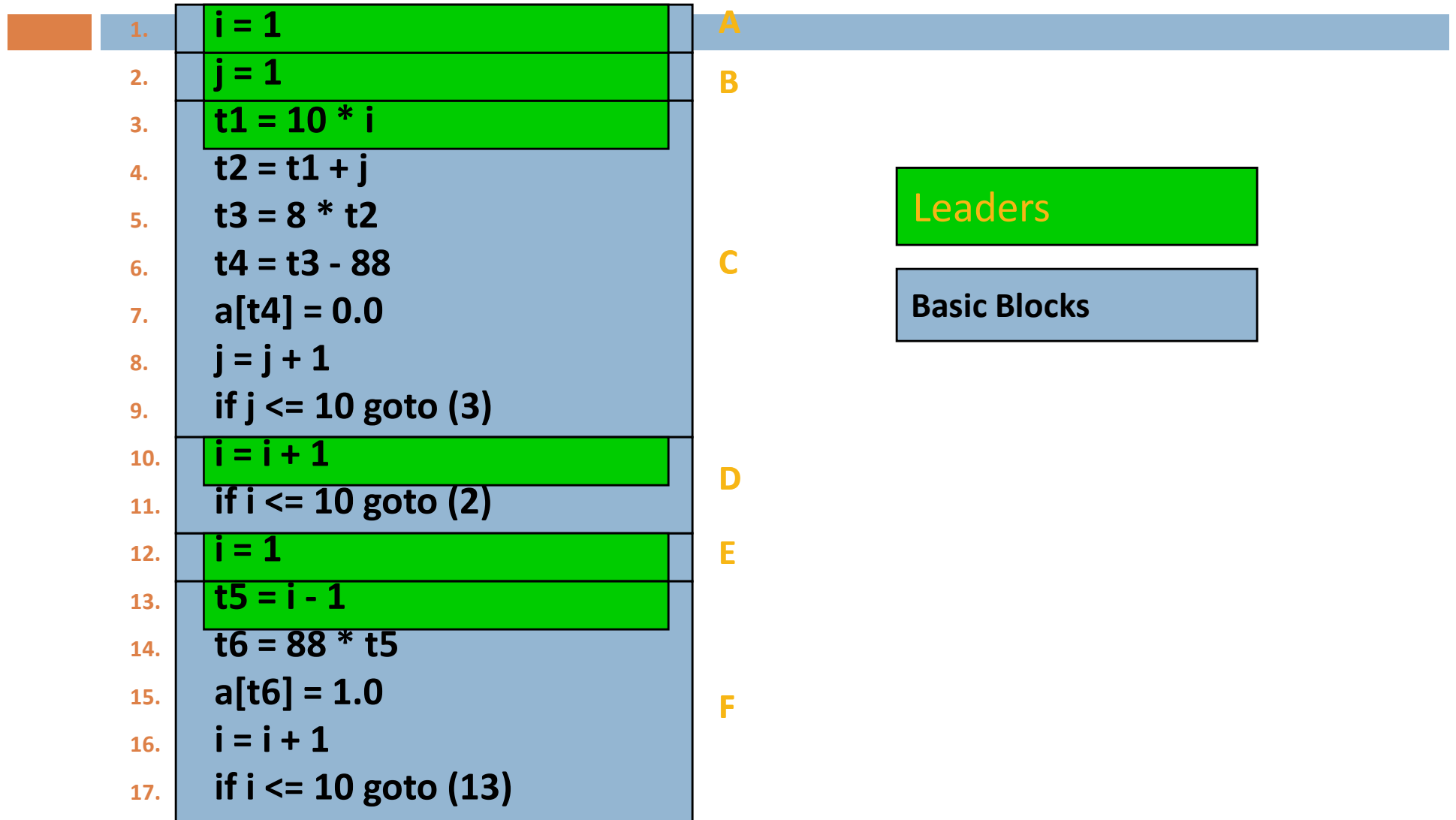
Identifying Basic Blocks

- Input: sequence of instructions *instr(i)*
- Output: A list of basic blocks
- Method:
 - ▣ Identify **leaders**:
the first instruction of a basic block
 - ▣ Iterate: add subsequent instructions to basic block
until we reach another leader

Identifying Leaders

- Rules for finding leaders in code
 - ▣ **First instr** in the code is a leader
 - ▣ Any instr that is the **target** of a (conditional or unconditional) jump is a leader
 - ▣ Any instr that **immediately follow** a (conditional or unconditional) jump is a leader

Basic Block Example



```

begin
    prod := 0;
    i := 1;
    do begin
        prod := prod + a[i] * b[i];
        i := i + 1
    end
    while i <= 20
end

```

```

(1)  prod := 0;
(2)  i := 1;
(3)  t1 := 4 * i  /* compute a[i] */
(4)  t2 := a[t1];
(5)  t3 := 4 * i  /* compute b[i] */
(6)  t4 := b[t3];
(7)  t5 := t2 * t4;
(8)  t6 := prod + t5;
(9)  prod := t6;
(10) t7 := i + 1;
(11) i := t7;
(12) if i <= 20 goto (3)

```

Fig. 9.8. Three-address code computing dot product.

Control Flow graphs

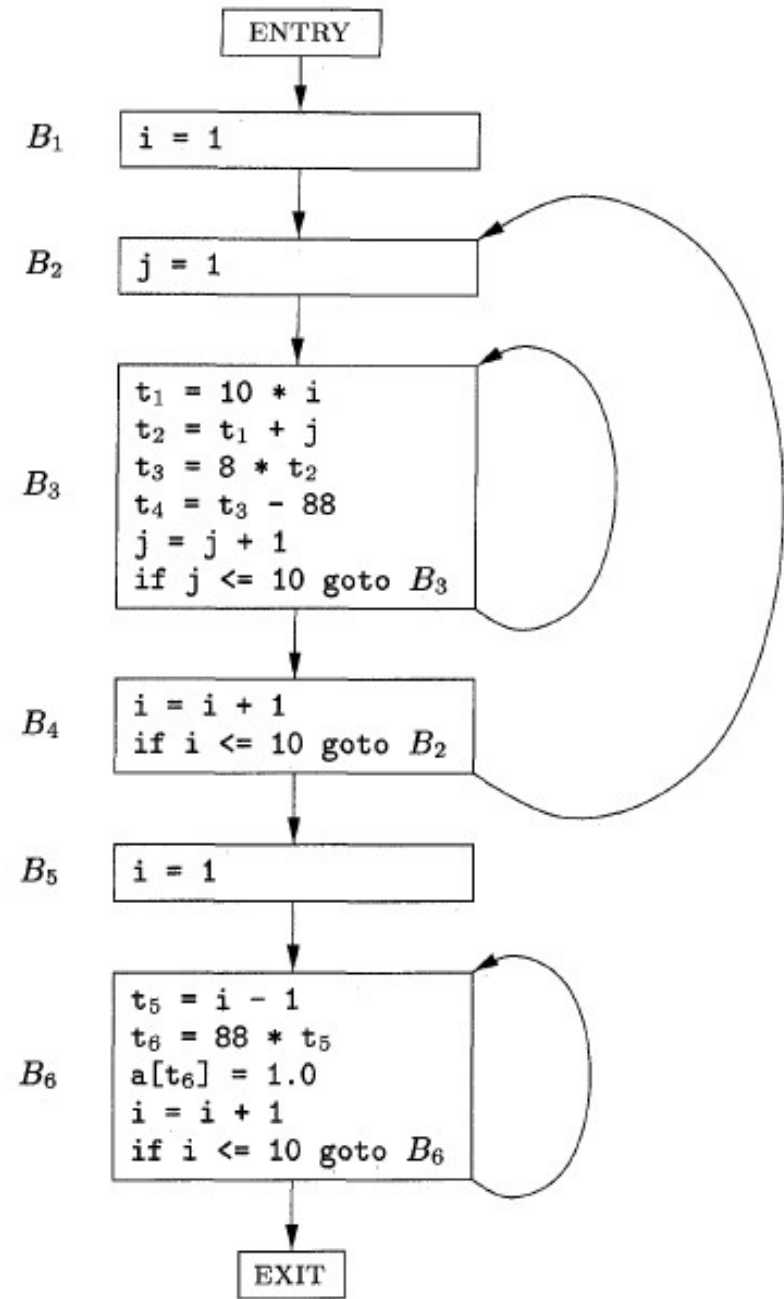


- add control flow information to basic blocks
- nodes are the basic blocks
- there is a directed edge from B_1 to B_2 if B_2 can follow B_1 in some execution sequence
 - there is a jump from the last statement of B_1 to the first statement of B_2
 - B_2 follows B_1 in natural order of execution
- initial node: block with first statement as leader

Control-Flow Edges

- Basic blocks = nodes
- Edges:
 - ▣ Add directed edge between B1 and B2 if:
 - Branch from last statement of B1 to first statement of B2 (B2 is a leader), or
 - B2 immediately follows B1 in program order and B1 does not end with unconditional branch (goto)
 - ▣ Definition of predecessor and successor
 - B1 is a predecessor of B2
 - B2 is a successor of B1

CFG Example



Find basic block and Flow graph

1. Convert the given code into Intermediate code
2. Find basic blocks and draw control flow graph

```
1:  sum=0
2:  i=1
3:  while ( i<N) do
4:      i=i+1
5:      sum=sum+i
    endwhile
6:  print(sum)
```

Find basic block and Flow graph

3 Address code

```
1: sum = 0
2: i = 1
3: if i >= N goto (9)
4: t1 = i + 1
5: i = t1
6: t2 = sum + i
7: sum = t2
8: goto (3)
9: print sum
```

Find Leader

```
L1: Line 1
L2: Line 3
L3: Line 4
L4: Line 9
```

Basic Blocks

B1

```
1: sum = 0
2: i = 1
```

B2

```
3: if i >= N goto B4
```

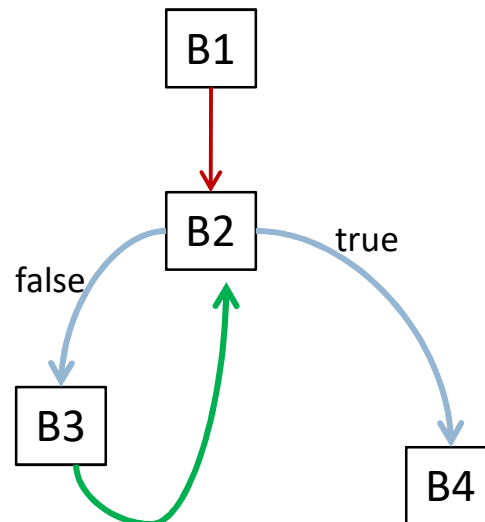
B3

```
4: t1 = i + 1
5: i = t1
6: t2 = sum + i
7: sum = t2
8: goto B2
```

B4

```
9: print sum
```

Control Flow Graph



Next use information

- for register and temporary allocation
- remove variables from registers if not used
- statement **$X = Y \text{ op } Z$**
defines X and uses Y and Z
- scan each basic blocks backwards
- assume all temporaries are dead on exit and all user variables are live on exit

Computing next use information



Suppose we are scanning

$i : X := Y \text{ op } Z$

in backward scan

1. attach to statement **i** , information in symbol table about **X, Y, Z**
2. set **X** to “not live” and “no next use” in symbol table
3. set **Y** and **Z** to be “live” and next use as **i** in symbol table

Example

$$1: t_1 = a * a$$

$$2: t_2 = a * b$$

$$3: t_3 = 2 * t_2$$

$$4: t_4 = t_1 + t_3$$

$$5: t_5 = b * b$$

$$6: t_6 = t_4 + t_5$$

$$7: X = t_6$$

STATEMENT

1: $t_1 = a * a$
2: $t_2 = a * b$
3: $t_3 = 2 * t_2$
4: $t_4 = t_1 + t_3$
5: $t_5 = b * b$
6: $t_6 = t_4 + t_5$
7: $X = t_6$

Example

Symbol Table

t_1	dead	Use in 4
t_2	dead	Use in 3
t_3	dead	Use in 4
t_4	dead	Use in 6
t_5	dead	Use in 6
t_6	dead	Use in 7

7: no temporary is live
6: t_6 :use(7), t_4 t_5 not live
5: t_5 :use(6)
4: t_4 :use(6), t_1 t_3 not live
3: t_3 :use(4), t_2 not live
2: t_2 :use(3)
1: t_1 :use(4)

Example ...

STATEMENT

1: $t_1 = a * a$

2: $t_2 = a * b$

3: $t_3 = 2 * t_2$

4: $t_4 = t_1 + t_3$

5: $t_5 = b * b$

6: $t_6 = t_4 + t_5$

7: $X = t_6$

1

2

3

4

5

6

7

t_1

t_2

t_3

t_4

t_5

t_6

1: $t_1 = a * a$

2: $t_2 = a * b$

3: $t_2 = 2 * t_2$

4: $t_1 = t_1 + t_2$

5: $t_2 = b * b$

6: $t_1 = t_1 + t_2$

7: $X = t_1$

Code Generator



- consider each statement
- remember if operands are in registers
- **Register descriptor**
 - Keep track of what is currently in each register.
 - Initially all the registers are empty
- **Address descriptor**
 - Keep track of location where current value of the name can be found at runtime
 - The location might be a register, stack, memory address or a set of those

Code Generation Algorithm

for each $X = Y \text{ op } Z$ do

- invoke a function **getreg** to determine location L where X must be stored. Usually L is a register.
- Consult address descriptor of Y to determine Y' . Prefer a register for Y' . If value of Y not already in L generate
Mov Y', L

Code Generation Algorithm

- Generate

$op\ Z', L$

Again prefer a register for Z. Update address descriptor of X to indicate X is in L.

- If L is a register, update its descriptor to indicate that it contains X and remove X from all other register descriptors.
- If current value of Y and/or Z have no next use and are dead on exit from block and are in registers, change register descriptor to indicate that they no longer contain Y and/or Z.

Function getreg

1. If Y is in register (that holds no other values) and Y is not live and has no next use after
$$X = Y \text{ op } Z$$
then return register of Y for L.
2. Failing (1) return an empty register
3. Failing (2) if X has a next use in the block or op requires register then get a register R, store its content into M (by Mov R, M) and use it.
4. else select memory location X as L

Example

28

$t_1 = a - b$
 $t_2 = a - c$
 $t_3 = t_1 + t_2$
 $d = t_3 + t_2$

Stmt

$t_1 = a - b$

code

mov a, R₀

sub b, R₀

reg desc

R₀ contains t_1

addr desc

t_1 in R₀

$t_2 = a - c$

mov a, R₁

sub c, R₁

R₀ contains t_1

t_1 in R₀

R₁ contains t_2

t_2 in R₁

$t_3 = t_1 + t_2$

add R₁, R₀

R₀ contains t_3

t_3 in R₀

R₁ contains t_2

t_2 in R₁

$d = t_3 + t_2$

add R₁, R₀

mov R₀, d

R₀ contains d

d in R₀

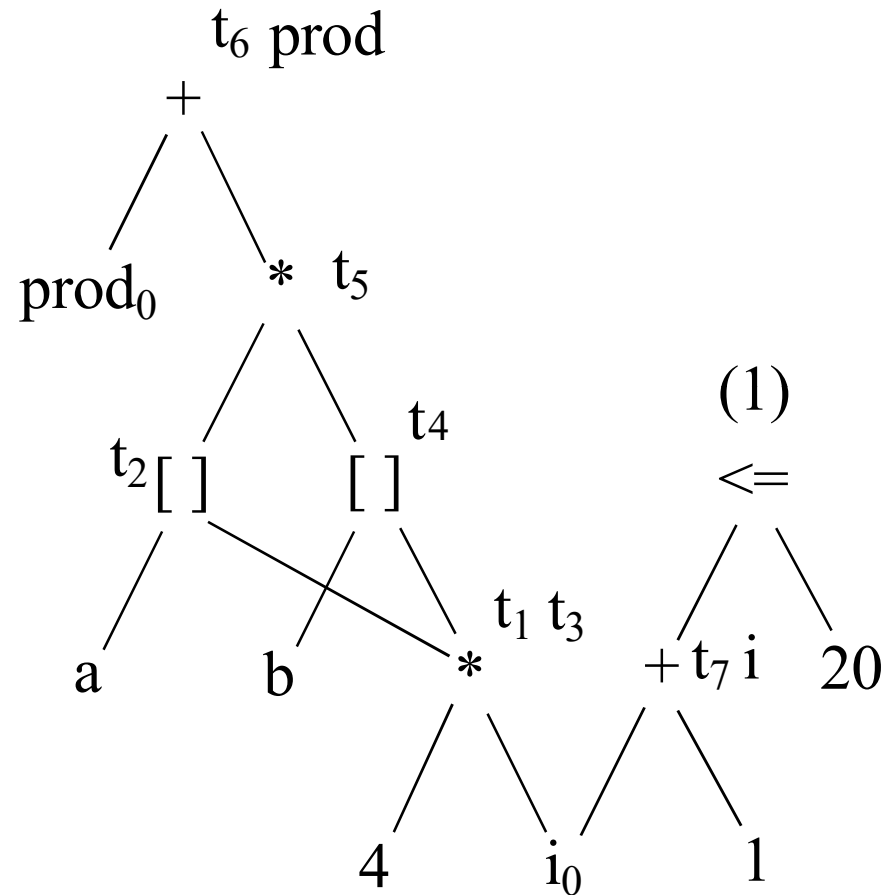
d in R₀ and
memory

DAG representation of basic blocks

- ⑩ useful data structures for implementing transformations on basic blocks
- ⑩ gives a picture of how value computed by a statement is used in subsequent statements
- ⑩ good way of determining common sub-expressions
- ⑩ A dag for a basic block has following labels on the nodes
 - leaves are labeled by unique identifiers, either variable names or constants
 - interior nodes are labeled by an operator symbol
 - nodes are also optionally given a sequence of identifiers for labels

DAG representation: example

1. $t_1 := 4 * i$
2. $t_2 := a[t_1]$
3. $t_3 := 4 * i$
4. $t_4 := b[t_3]$
5. $t_5 := t_2 * t_4$
6. $t_6 := \text{prod} + t_5$
7. $\text{prod} := t_6$
8. $t_7 := i + 1$
9. ~~if $t_7 \neq 20$ goto (1)~~
10. if $t_7 \neq 20$ goto (1)



Code Generation from DAG

$S_1 = 4 * i$

$S_2 = \text{addr}(A) - 4$

$S_3 = S_2[S_1]$

$S_4 = 4 * i$

$S_5 = \text{addr}(B) - 4$

$S_6 = S_5[S_4]$

$S_7 = S_3 * S_6$

$S_8 = \text{prod} + S_7$

$\text{prod} = S_8$

$S_9 = I + 1$

$I = S_9$

If $I \leq 20$ goto (1)

$S_1 = 4 * i$

$S_2 = \text{addr}(A) - 4$

$S_3 = S_2[S_1]$

$S_5 = \text{addr}(B) - 4$

$S_6 = S_5[S_4]$

$S_7 = S_3 * S_6$

$\text{prod} = \text{prod} + S_7$

$I = I + 1$

If $I \leq 20$ goto (1)

Rearranging order of the code

- Consider following basic block

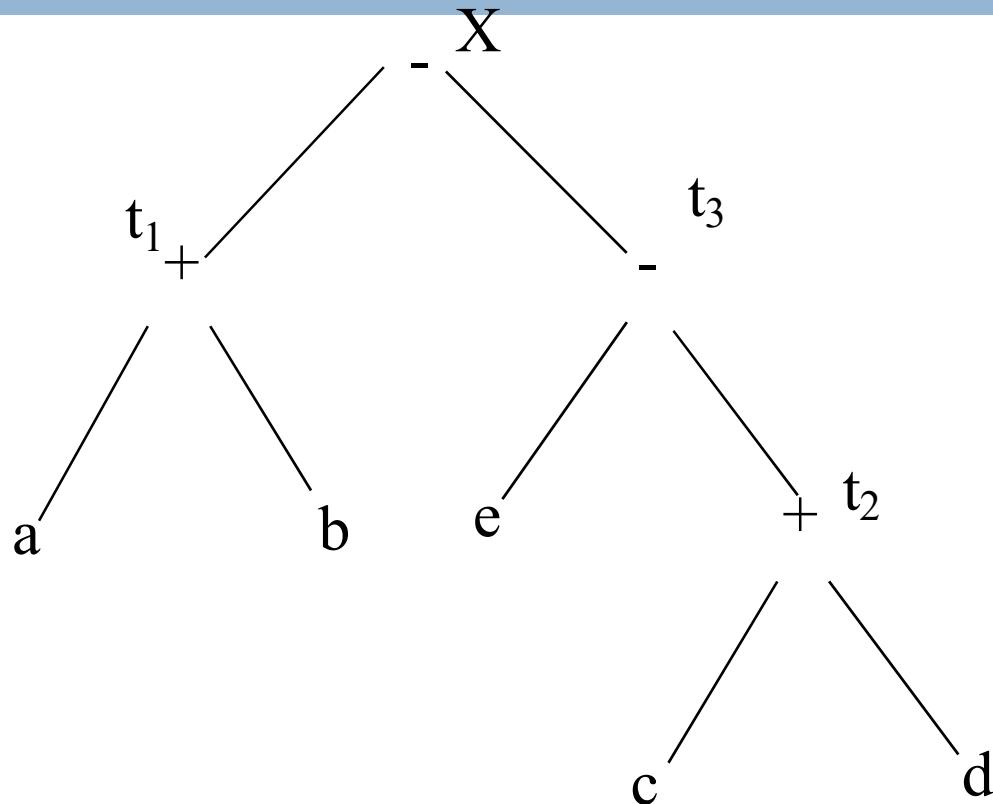
$$t_1 = a + b$$

$$t_2 = c + d$$

$$t_3 = e - t_2$$

$$X = t_1 - t_3$$

and its DAG



Rearranging order ...

Rearranging the code

Three address code for the DAG (assuming only two registers are available)

```
MOV a, R0
ADD b, R0
MOV c, R1
ADD d, R1
MOV R0, t1
MOV e, R0
SUB R1, R0
MOV t1, R1
SUB R0, R1
MOV R1, X
```

Register spilling

Register reloading

as $t_2 = c + d$

$t_3 = e - t_2$

$t_1 = a + b$

$X = t_1 - t_3$

gives

```
MOV c, R0
ADD d, R0
MOV e, R1
SUB R0, R1
MOV a, R0
ADD b, R0
SUB R1, R0
MOV R1, X
```